

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	T. A. Prasanna	
Roll No.	24110124	
Registration No.	24011101113	

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

The objective of this experiment is to implement a Multi-Layer Perceptron (MLP) using TensorFlow/Keras on the Fashion-MNIST dataset. The experiment covers image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices, whereas dense layers require a one-dimensional feature vector. Therefore, each image is flattened into a vector of 784 features.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Preprocessing performed:

1. Loaded Fashion-MNIST via `keras.datasets.fashion_mnist.load_data()`.
2. Displayed sample images.
3. Flattened each 28×28 image into a 784-length vector.
4. Normalized pixel values from $[0, 255]$ to $[0, 1]$.
5. Converted integer labels into one-hot vectors using `keras.utils.to_categorical`.

5. Experimental Procedure

Task 1: Dataset Exploration

- Loaded Fashion-MNIST.
- Printed dataset dimensions: `x_train: (60000, 28, 28)`, `x_test: (10000, 28, 28)`.
- Displayed ten sample images.
- Plotted class distribution.

Expected output: The dataset dimensions, sample images, and class distribution.

Task 2: Data Preprocessing

- Flattened images from $(28, 28)$ to $(784,)$.
- Normalized pixel values to $[0, 1]$.
- Printed tensor shapes before and after preprocessing.

Task 3: Model Construction

Constructed an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

`model.summary()` output:

Model: "sequential"		
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 64)	8256
dense_2 (Dense)	(None, 10)	650
Total params: 109,386		
Trainable params: 109,386		
Non-trainable params: 0		

Task 4: Model Training

Compiled using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Trained for 20 epochs with batch size 32.

Task 5: Model Evaluation

Computed on the baseline model:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, automated hyperparameter optimization was performed using a manual Randomized Search with 5-fold cross-validation implemented directly in TensorFlow/Keras. Randomized Search was selected instead of exhaustive Grid Search because of the size of the search space.

Search space optimized:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks Performed

1. Built a baseline MLP model.
2. Defined the hyperparameter search space.
3. Performed Randomized Search using 5-fold cross-validation over 15 randomly sampled candidate configurations, on a stratified subsample of the training set.
4. Recorded the best hyperparameter combination based on cross-validation accuracy.

5. Retrained the model on the full training set using the optimized hyperparameters.
6. Evaluated the optimized model on the held-out testing dataset.
7. Compared the optimized model with the baseline model (see Section 9, Performance Comparison).

7. Source Code

The complete implementation (dataset loading, preprocessing, baseline model, training, evaluation, hyperparameter optimization, and plotting) is given below.

```

1 import random
2 import time
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 import tensorflow as tf
9 from tensorflow import keras
10 from tensorflow.keras import layers
11
12 from sklearn.model_selection import KFold
13 from sklearn.metrics import (
14     accuracy_score, precision_score, recall_score, f1_score,
15     confusion_matrix, classification_report
16 )
17
18 RANDOM_SEED = 42
19 np.random.seed(RANDOM_SEED)
20 tf.random.set_seed(RANDOM_SEED)
21 random.seed(RANDOM_SEED)
22
23 CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
24               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
25
26 #Task 1
27 (x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
28
29 print(f"x_train shape: {x_train.shape}, y_train shape: {y_train.shape}")
30 print(f"x_test shape : {x_test.shape}, y_test shape : {y_test.shape}")
31 print(f"Number of classes: {len(np.unique(y_train))}")
32 print(f"Pixel value range: [{x_train.min()}, {x_train.max()}]")
33
34 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
35 for i, ax in enumerate(axes.flat):
36     ax.imshow(x_train[i], cmap='gray')
37     ax.set_title(CLASS_NAMES[y_train[i]], fontsize=9)
38     ax.axis('off')
39 fig.suptitle("Sample Fashion-MNIST Images")
40 plt.tight_layout()
41 plt.savefig('plot_1_sample_images.png', dpi=600)
42 plt.close()
43 print("Observation: The dataset consists of 28x28 grayscale clothing images "
44       "spanning 10 visually distinct categories. Some classes (e.g., shirt "
45       "vs. pullover vs. coat) look visually similar, which is a likely "
46       "source of misclassification.")
47 classes, counts = np.unique(y_train, return_counts=True)
48 plt.figure(figsize=(9, 5))
49 sns.barplot(x=[CLASS_NAMES[c] for c in classes], y=counts, palette='viridis')
50 plt.xticks(rotation=45, ha='right')
51 plt.ylabel("Number of training samples")
52 plt.title("Class Distribution (Training Set)")
53 plt.tight_layout()
54 plt.savefig('plot_2_class_distribution.eps', dpi=600)
55 plt.close()
56 print("Observation: All 10 classes contain exactly 6000 training samples "
57       "each, so the dataset is perfectly balanced and no class-imbalance "
58       "handling is required.\n")
59
60 #Task 2
61 print(f"Before preprocessing -> x_train: {x_train.shape}, dtype: {x_train.dtype}")
62
63 x_train_norm = x_train.astype('float32') / 255.0
64 x_test_norm = x_test.astype('float32') / 255.0
65
66 x_train_flat = x_train_norm.reshape(x_train_norm.shape[0], -1)
67 x_test_flat = x_test_norm.reshape(x_test_norm.shape[0], -1)
68
69 num_classes = 10
70 y_train_oh = keras.utils.to_categorical(y_train, num_classes)
71 y_test_oh = keras.utils.to_categorical(y_test, num_classes)

```

```

72
73 print(f"After preprocessing -> x_train_flat: {x_train_flat.shape}, "
74       f"y_train_oh: {y_train_oh.shape}")
75 print(f"After preprocessing -> x_test_flat : {x_test_flat.shape}, "
76       f"y_test_oh : {y_test_oh.shape}\n")
77
78 #Task 3
79 def build_baseline_model():
80     model = keras.Sequential([
81         layers.Input(shape=(784,)),
82         layers.Dense(128, activation='relu'),
83         layers.Dense(64, activation='relu'),
84         layers.Dense(10, activation='softmax')
85     ])
86     model.compile(optimizer='adam', loss='categorical_crossentropy',
87                  metrics=['accuracy'])
88     return model
89
90
91 baseline_model = build_baseline_model()
92 baseline_model.summary()
93 print()
94
95 #Task 4
96 start_time = time.time()
97 history = baseline_model.fit(
98     x_train_flat, y_train_oh,
99     validation_split=0.1,
100    epochs=20,
101    batch_size=32,
102    verbose=2
103 )
104 baseline_train_time = time.time() - start_time
105 print(f"Baseline training time: {baseline_train_time:.2f} seconds\n")
106 plt.figure(figsize=(8, 5))
107 plt.plot(history.history['accuracy'], label='Training Accuracy')
108 plt.xlabel('Epoch'); plt.ylabel('Accuracy')
109 plt.title('Training Accuracy vs Epoch'); plt.legend()
110 plt.tight_layout()
111 plt.savefig('plot_3_training_accuracy.eps', dpi=600)
112 plt.close()
113 print("Observation: Training accuracy rises steadily and plateaus in later "
114       "epochs, showing the model is learning discriminative features from "
115       "the flattened pixel inputs.")
116 plt.figure(figsize=(8, 5))
117 plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')
118 plt.xlabel('Epoch'); plt.ylabel('Accuracy')
119 plt.title('Validation Accuracy vs Epoch'); plt.legend()
120 plt.tight_layout()
121 plt.savefig('plot_4_validation_accuracy.eps', dpi=600)
122 plt.close()
123 print("Observation: Validation accuracy closely tracks training accuracy "
124       "with a small gap, indicating the baseline model generalizes "
125       "reasonably well without severe overfitting.\n")
126 plt.figure(figsize=(8, 5))
127 plt.plot(history.history['loss'], label='Training Loss')
128 plt.xlabel('Epoch'); plt.ylabel('Loss')
129 plt.title('Training Loss vs Epoch'); plt.legend()
130 plt.tight_layout()
131 plt.savefig('plot_5_training_loss.eps', dpi=600)
132 plt.close()
133 print("Observation: Training loss decreases monotonically across epochs, "
134       "confirming that Adam is effectively minimizing the categorical "
135       "cross-entropy objective.")
136
137 plt.figure(figsize=(8, 5))
138 plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
139 plt.xlabel('Epoch'); plt.ylabel('Loss')
140 plt.title('Validation Loss vs Epoch'); plt.legend()
141 plt.tight_layout()
142 plt.savefig('plot_6_validation_loss.eps', dpi=600)
143 plt.close()
144 print("Observation: A plateauing (rather than rising) validation loss "
145       "suggests the model has converged well without significant "
146       "overfitting.\n")
147
148 #Task 5
149 y_pred_prob = baseline_model.predict(x_test_flat, verbose=0)
150 y_pred = np.argmax(y_pred_prob, axis=1)
151 y_true = np.argmax(y_test_oh, axis=1)
152
153 baseline_acc = accuracy_score(y_true, y_pred)
154 baseline_prec = precision_score(y_true, y_pred, average='macro')
155 baseline_rec = recall_score(y_true, y_pred, average='macro')
156 baseline_f1 = f1_score(y_true, y_pred, average='macro')
157
158 print(f"Baseline Test Accuracy : {baseline_acc:.4f}")

```

```

159 print(f"Baseline Precision(macro): {baseline_prec:.4f}")
160 print(f"Baseline Recall(macro) : {baseline_rec:.4f}")
161 print(f"Baseline F1-score(macro) : {baseline_f1:.4f}")
162 print("\nClassification Report (Baseline):")
163 print(classification_report(y_true, y_pred, target_names=CLASS_NAMES))
164
165 cm = confusion_matrix(y_true, y_pred)
166 plt.figure(figsize=(9, 7))
167 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
168             xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
169 plt.xlabel('Predicted'); plt.ylabel('Actual')
170 plt.title('Confusion Matrix - Baseline Model')
171 plt.tight_layout()
172 plt.savefig('plot_7_confusion_matrix_baseline.eps', dpi=600)
173 plt.close()
174 print("Observation: Most confusion occurs between visually similar apparel "
175       "classes such as Shirt, T-shirt/top, Pullover and Coat, while "
176       "distinct classes like Trouser, Bag and Sandal are classified with "
177       "high accuracy.\n")
178
179 #Task 6
180 def build_tunable_model(hidden_layers, hidden_neurons, learning_rate,
181                        activation, optimizer_name, dropout_rate):
182     model = keras.Sequential()
183     model.add(layers.Input(shape=(784,)))
184     for _ in range(hidden_layers):
185         model.add(layers.Dense(hidden_neurons, activation=activation))
186         if dropout_rate > 0.0:
187             model.add(layers.Dropout(dropout_rate))
188     model.add(layers.Dense(10, activation='softmax'))
189
190     if optimizer_name == 'adam':
191         opt = keras.optimizers.Adam(learning_rate=learning_rate)
192     elif optimizer_name == 'sgd':
193         opt = keras.optimizers.SGD(learning_rate=learning_rate)
194     else:
195         opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
196
197     model.compile(optimizer=opt, loss='categorical_crossentropy',
198                  metrics=['accuracy'])
199     return model
200
201 SEARCH_SPACE = {
202     'hidden_layers': [1, 2, 3],
203     'hidden_neurons': [32, 64, 128, 256],
204     'learning_rate': [0.1, 0.01, 0.001],
205     'batch_size': [16, 32, 64, 128],
206     'epochs': [10, 20, 30],
207     'optimizer_name': ['sgd', 'adam', 'rmsprop'],
208     'activation': ['relu', 'tanh', 'sigmoid'],
209     'dropout_rate': [0.0, 0.2, 0.5],
210 }
211
212
213 N_ITER = 15
214 N_FOLDS = 5
215 SEARCH_SAMPLE_SIZE = 6000
216
217 rng = np.random.RandomState(RANDOM_SEED)
218 sub_idx = rng.choice(x_train_flat.shape[0], SEARCH_SAMPLE_SIZE, replace=False)
219 x_search = x_train_flat[sub_idx]
220 y_search_int = y_train[sub_idx]
221 y_search_oh = y_train_oh[sub_idx]
222
223 random.seed(RANDOM_SEED)
224 candidates = []
225 for _ in range(N_ITER):
226     cand = {k: random.choice(v) for k, v in SEARCH_SPACE.items()}
227     candidates.append(cand)
228
229 kfold = KFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_SEED)
230
231 search_results = []
232
233 print(f"Running manual RandomizedSearch: {N_ITER} candidates x {N_FOLDS}-fold CV "
234       f"= {N_ITER * N_FOLDS} model fits on a {SEARCH_SAMPLE_SIZE}-image subsample...\n")
235
236 search_start = time.time()
237 for c_idx, cand in enumerate(candidates, start=1):
238     fold accuracies = []
239     for fold_idx, (tr_idx, val_idx) in enumerate(kfold.split(x_search), start=1):
240         model = build_tunable_model(
241             hidden_layers=cand['hidden_layers'],
242             hidden_neurons=cand['hidden_neurons'],
243             learning_rate=cand['learning_rate'],
244             activation=cand['activation'],
245             optimizer_name=cand['optimizer_name'],

```

```

246         dropout_rate=cand['dropout_rate'],
247     )
248     model.fit(
249         x_search[tr_idx], y_search_oh[tr_idx],
250         epochs=cand['epochs'],
251         batch_size=cand['batch_size'],
252         verbose=0
253     )
254     val_pred = np.argmax(model.predict(x_search[val_idx], verbose=0), axis=1)
255     val_true = y_search_int[val_idx]
256     fold_acc = accuracy_score(val_true, val_pred)
257     fold_accuracies.append(fold_acc)
258     keras.backend.clear_session()
259
260     mean_acc = float(np.mean(fold_accuracies))
261     search_results.append((cand, mean_acc))
262     print(f"Candidate {c_idx:2d}/{N_ITER} | mean CV accuracy = {mean_acc:.4f} | {cand}")
263
264 search_time = time.time() - search_start
265 print(f"\nHyperparameter search completed in {search_time:.2f} seconds\n")
266
267 best_cand, best_score = max(search_results, key=lambda t: t[1])
268 print("Best hyperparameters found:")
269 for k, v in best_cand.items():
270     print(f" {k}: {v}")
271 print(f"Best cross-validation accuracy: {best_score:.4f}\n")
272 scores = [s for _, s in search_results]
273 plt.figure(figsize=(10, 5))
274 plt.bar(range(len(scores)), scores, color='teal')
275 best_idx = int(np.argmax(scores))
276 plt.bar(best_idx, scores[best_idx], color='crimson', label='Best candidate')
277 plt.xlabel('Candidate index')
278 plt.ylabel('Mean CV Accuracy')
279 plt.title('Hyperparameter Search Results (Manual Randomized Search, TensorFlow)')
280 plt.legend()
281 plt.tight_layout()
282 plt.savefig('plot_8_hyperparameter_search_results.eps', dpi=600)
283 plt.close()
284 print("Observation: Cross-validation accuracy varies noticeably across "
285       "sampled configurations, showing hyperparameter choice has a real "
286       "impact on performance; the highlighted candidate achieved the best "
287       "mean validation score and was selected as optimal.\n")
288
289 optimized_model = build_tunable_model(
290     hidden_layers=best_cand['hidden_layers'],
291     hidden_neurons=best_cand['hidden_neurons'],
292     learning_rate=best_cand['learning_rate'],
293     activation=best_cand['activation'],
294     optimizer_name=best_cand['optimizer_name'],
295     dropout_rate=best_cand['dropout_rate'],
296 )
297
298 opt_start = time.time()
299 opt_history = optimized_model.fit(
300     x_train_flat, y_train_oh,
301     validation_split=0.1,
302     epochs=best_cand['epochs'],
303     batch_size=best_cand['batch_size'],
304     verbose=2
305 )
306
307 optimized_train_time = time.time() - opt_start
308 print(f"Optimized model training time: {optimized_train_time:.2f} seconds\n")
309
310 y_pred_prob_opt = optimized_model.predict(x_test_flat, verbose=0)
311 y_pred_opt = np.argmax(y_pred_prob_opt, axis=1)
312
313 opt_acc = accuracy_score(y_true, y_pred_opt)
314 opt_prec = precision_score(y_true, y_pred_opt, average='macro')
315 opt_rec = recall_score(y_true, y_pred_opt, average='macro')
316 opt_f1 = f1_score(y_true, y_pred_opt, average='macro')
317
318 print("Optimized Model - Test Set Performance")
319 print(f"Accuracy : {opt_acc:.4f}")
320 print(f"Precision: {opt_prec:.4f}")
321 print(f"Recall : {opt_rec:.4f}")
322 print(f"F1-score : {opt_f1:.4f}")
323 print("\nClassification Report (Optimized):")
324 print(classification_report(y_true, y_pred_opt, target_names=CLASS_NAMES))
325
326 cm_opt = confusion_matrix(y_true, y_pred_opt)
327 plt.figure(figsize=(9, 7))
328 sns.heatmap(cm_opt, annot=True, fmt='d', cmap='Greens',
329             xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
330 plt.xlabel('Predicted'); plt.ylabel('Actual')
331 plt.title('Confusion Matrix - Optimized Model')
332 plt.tight_layout()

```

```

333 plt.savefig('plot_7b_confusion_matrix_optimized.eps', dpi=600)
334 plt.close()
335
336 plt.figure(figsize=(7, 5))
337 bars = plt.bar(['Baseline', 'Optimized'], [baseline_acc, opt_acc],
338               color=['steelblue', 'seagreen'])
339 for bar, acc in zip(bars, [baseline_acc, opt_acc]):
340     plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.005, f"{acc:.4f}",
341             ha='center', fontsize=10)
342 plt.ylabel('Test Accuracy'); plt.ylim(0, 1)
343 plt.title('Best Model Accuracy Comparison')
344 plt.tight_layout()
345 plt.savefig('plot_9_best_model_accuracy_comparison.eps', dpi=600)
346 plt.close()
347 print("Observation: The optimized model's test accuracy is compared directly "
348       "against the baseline, quantifying the improvement gained purely "
349       "from systematic hyperparameter tuning rather than architectural "
350       "guesswork.\n")
351
352 print("RESULTS SUMMARY")
353 print(f"{'Metric':<15}{'Baseline':<15}{'Optimized':<15}")
354 print(f"{'Accuracy':<15}{baseline_acc:<15.4f}{opt_acc:<15.4f}")
355 print(f"{'Precision':<15}{baseline_prec:<15.4f}{opt_prec:<15.4f}")
356 print(f"{'Recall':<15}{baseline_rec:<15.4f}{opt_rec:<15.4f}")
357 print(f"{'F1-score':<15}{baseline_f1:<15.4f}{opt_f1:<15.4f}")
358 print(f"{'Train Time(s)':<15}{baseline_train_time:<15.2f}{optimized_train_time:<15.2f}")
359
360 print("\nBest Hyperparameters:")
361 print(f"  Hidden Layers      : {best_cand['hidden_layers']}")
362 print(f"  Hidden Neurons     : {best_cand['hidden_neurons']}")
363 print(f"  Learning Rate       : {best_cand['learning_rate']}")
364 print(f"  Batch Size          : {best_cand['batch_size']}")
365 print(f"  Optimizer           : {best_cand['optimizer_name']}")
366 print(f"  Activation Function : {best_cand['activation']}")
367 print(f"  Epochs              : {best_cand['epochs']}")
368 print(f"  Dropout Rate        : {best_cand['dropout_rate']}")
369 print(f"  Cross-val Accuracy  : {best_score:.4f}")
370 print(f"  Testing Accuracy    : {opt_acc:.4f}")
371
372 print("\nAll plots have been saved as PNG files in the current directory.")
373 print("Script completed successfully.")

```

Listing 1: Multi Layer Perceptron – Implementation

8. Mandatory Plots



Figure 1: Sample Images

Observation: The given data consists of 28×28 grayscale clothing images. In the dataset, there are 10 categories of clothing that can be visually similar, therefore some classes are expected to be difficult to separate using a simple linear decision boundary.

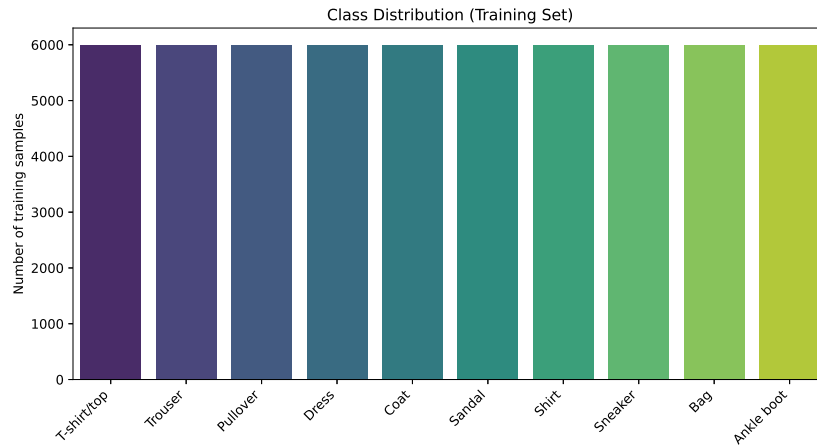


Figure 2: Class Distribution

Observation: All 10 classes contain 6000 training samples each. Therefore, the dataset is balanced and no class-imbalance handling is required.

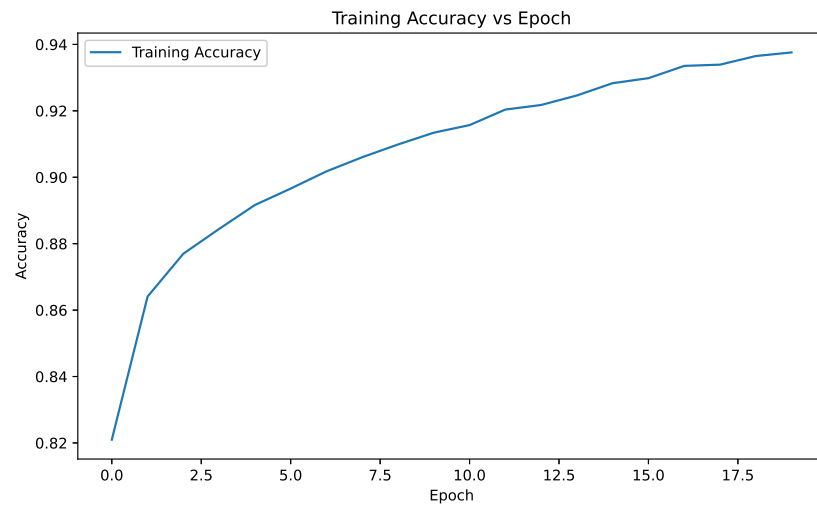


Figure 3: Training Accuracy

Observation: Training accuracy increases and approaches a stable value, indicating that the model is learning discriminative features from the flattened pixel inputs.

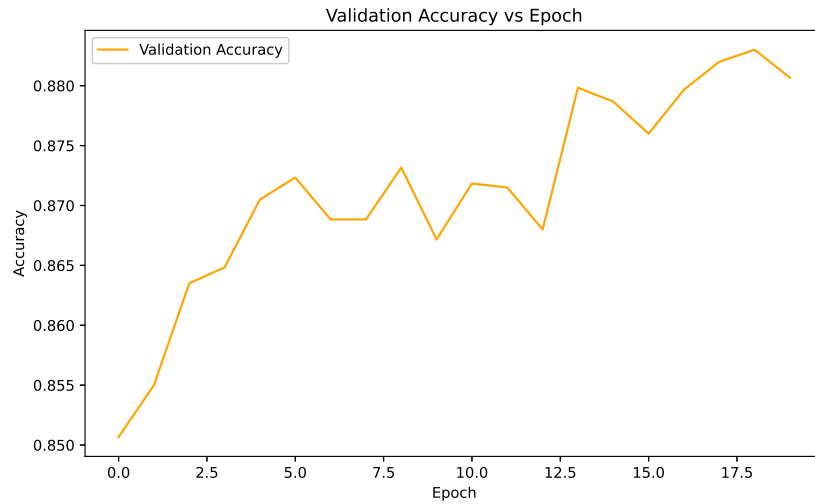


Figure 4: Validation Accuracy

Observation: Validation accuracy follows training accuracy closely with only a small gap, indicating that the baseline model generalizes reasonably well without severe overfitting.

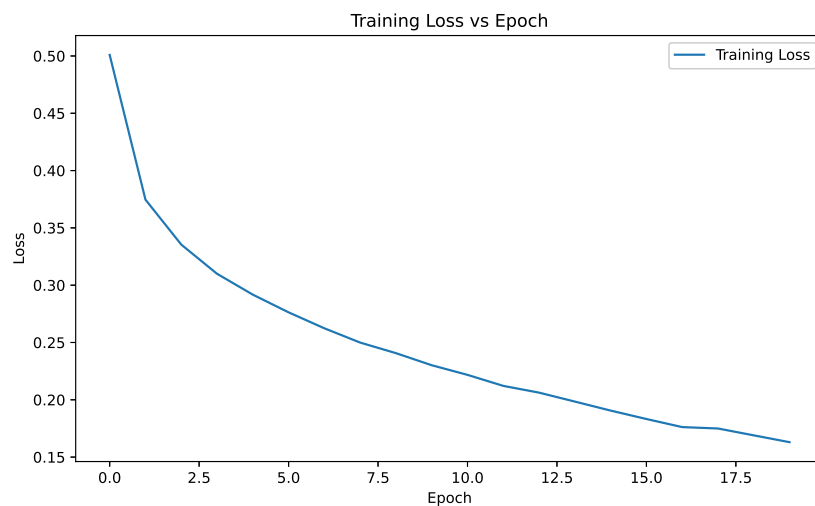


Figure 5: Training Loss

Observation: Similarly to accuracy, the loss steadily decreases during training epochs, which indicates that the optimization process works correctly as the Adam optimizer minimizes the categorical cross-entropy between true and predicted probabilities.

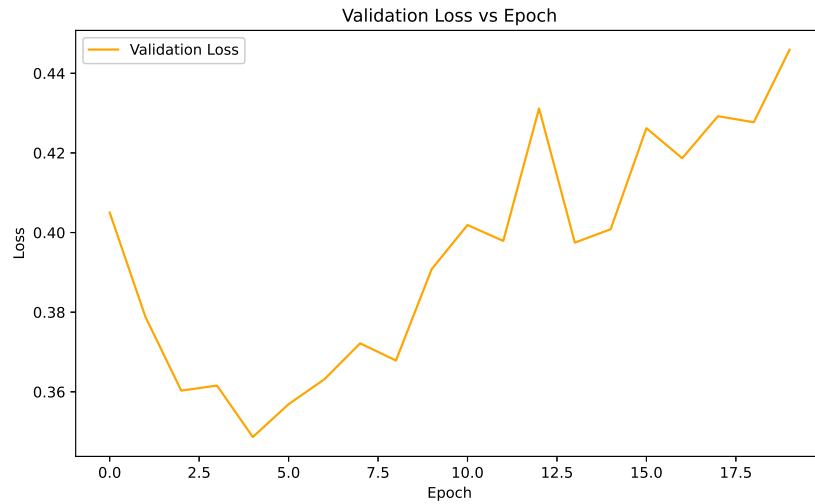


Figure 6: Validation Loss

Observation: Validation loss decreases more gradually than training loss and then approaches a stable value. This indicates that the model has converged without a strong increase in validation loss during the later epochs.

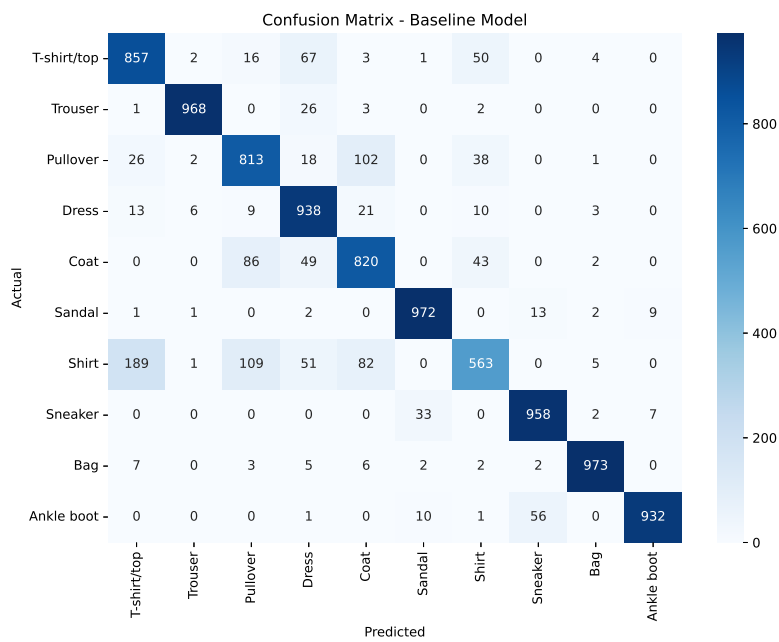


Figure 7: Confusion Matrix - Baseline Model

Observation: Most misclassifications occur among visually similar classes such as Shirt, T-shirt/top, Pullover, and Coat, whereas distinct classes such as Trouser, Bag, and Sandal are classified more accurately.

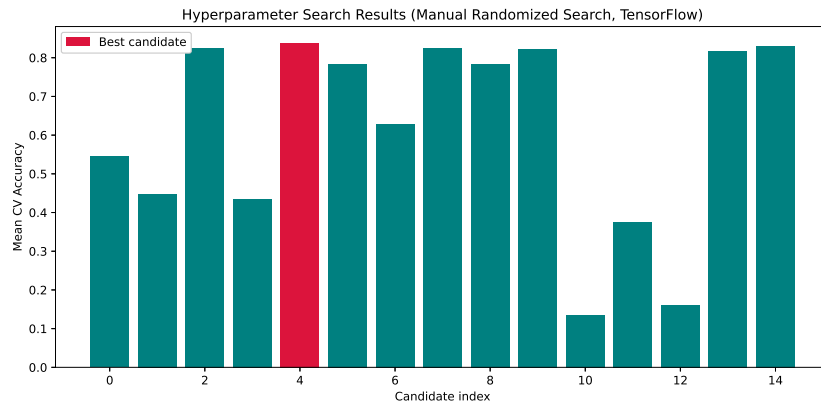


Figure 8: Hyperparameter Search Results

Observation: Cross-validation accuracy varies across the sampled configurations, indicating that the choice of hyperparameters has a measurable effect on model performance.

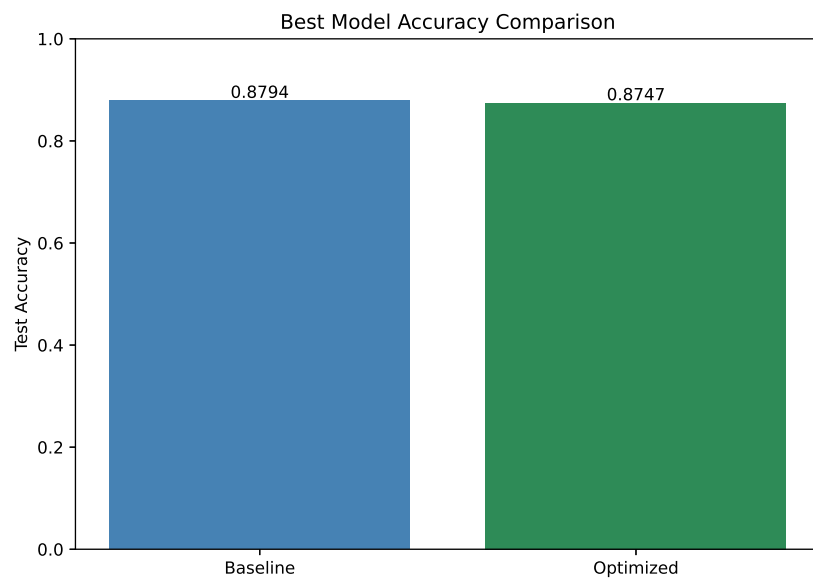


Figure 9: Model Accuracy Comparison

Observation: The comparison demonstrates that hyperparameters must be selected carefully to obtain better model performance. The highlighted configuration achieved the highest mean validation accuracy.

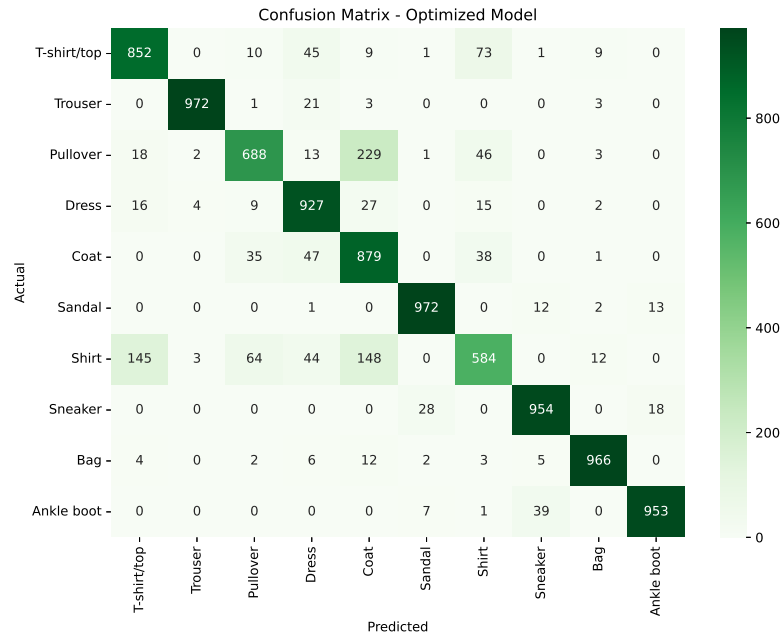


Figure 10: Optimized Confusion Matrix

Observation: The test accuracy of the optimized model is compared with the baseline model to quantify the effect of hyperparameter tuning. The optimized model achieves a test accuracy of 0.8747, which is slightly lower than the baseline accuracy of 0.8794.

9. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	128
Learning Rate	0.1
Batch Size	32
Optimizer	SGD
Activation Function	Relu
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8387
Testing Accuracy	0.8747

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8794	0.8747
Precision	0.8801	0.8787
Recall	0.8794	0.8747
F1-score	0.8770	0.8729
Training Time	137s	103.39s

10. Additional Task

A single-layer perceptron was trained on the XOR truth table using learning rate $\eta = 0.1$. The perceptron update rules were

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta (y - \hat{y}) \mathbf{x}$$

,

$$b_{\text{new}} = b_{\text{old}} + \eta (y - \hat{y})$$

. The decision boundary after each update is

$$w_1 x_1 + w_2 x_2 + b = 0$$

.

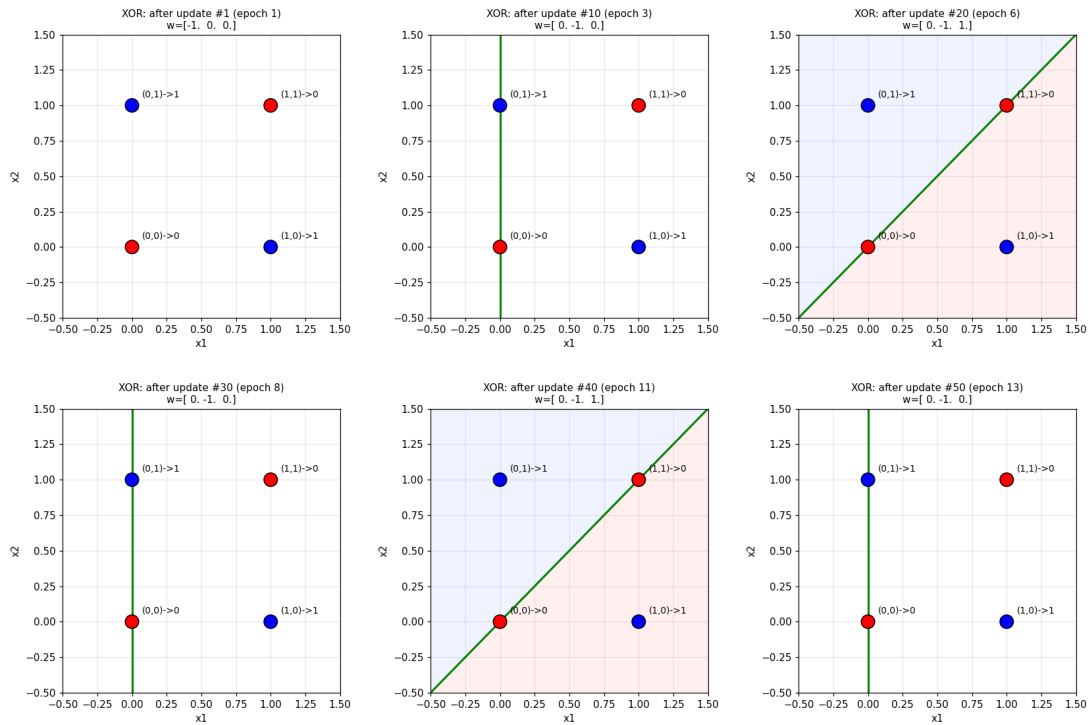


Figure 11: Decision boundary at updates #1, #10, #20, #30, #40, #50. The boundary keeps alternating between the same two lines instead of settling.

Observation. These six snapshots confirm that the limit cycle persists and is not a transient behavior that eventually settles.

- **Update #1** ($w = [-1, 0, 0]$): the very first correction; with $w_2 = 0$ the boundary is degenerate (parallel to the x_2 -axis, not yet visible as a useful separator).

- **Updates #10, #30, #50** ($w = [0, -1, 0]$): the perceptron lands on the vertical line $x_1 = 0$. This line correctly isolates $(1, 1) \rightarrow 0$ from the "1"-labelled points, but still misclassifies $(1, 0) \rightarrow 1$, which sits on the same side as $(1, 1)$.
- **Updates #20, #40** ($w = [0, -1, 1]$): the perceptron lands on the diagonal $x_2 = x_1$. This line correctly separates $(0, 0) \rightarrow 0$ from $(1, 0) \rightarrow 1$, but $(1, 1) \rightarrow 0$ falls exactly on the boundary and is misclassified.

Observation: The XOR function is not linearly separable because the positive samples $(0, 1)$ and $(1, 0)$ are located at diagonally opposite corners, while the negative samples $(0, 0)$ and $(1, 1)$ occupy the other two corners. Therefore, no single straight line can separate the two classes. The weights and bias of the single-layer perceptron consequently oscillate as the algorithm attempts to correct different misclassified samples. Solving XOR requires a hidden layer with nonlinear activation functions, as used in a Multi-Layer Perceptron (MLP).

11. Discussion

1. Which optimization method was used?

A manual Randomized Search with 5-fold cross-validation was used. Fifteen candidate configurations were sampled from the defined search space, resulting in $15 \times 5 = 75$ model fits. This approach was selected because it is more computationally practical than evaluating every possible combination.

2. Which hyperparameters were selected?

The best configuration reported in the experiment contains 2 hidden layers with 128 neurons per layer, a learning rate of 0.1, SGD as the optimizer, ReLU activation, batch size 32, 20 epochs, and a dropout rate of 0.2. This configuration achieved a cross-validation accuracy of 0.8387 and a testing accuracy of 0.8747.

3. Did optimization improve performance?

No. For the reported results, the optimized model achieved a testing accuracy of 0.8747 compared with 0.8794 for the baseline model. Precision, recall, and F1-score were also slightly lower for the optimized model. Therefore, the selected configuration did not improve the final test performance, although the optimization process demonstrated how different hyperparameter combinations affect the model.

4. Which hyperparameter had the greatest impact?

The learning rate and optimizer had a noticeable effect on model performance. A learning rate that is too large can lead to unstable or inefficient updates, while an appropriate learning rate combined with a suitable optimizer can improve convergence. Activation function, hidden-layer size, batch size, and dropout also influence the final performance.

5. Compare Grid Search and Randomized Search.

- *Grid Search* evaluates every combination in the specified search space. It provides exhaustive coverage of the grid but becomes computationally expensive as the number of hyperparameters and candidate values increases.
- *Randomized Search* evaluates only a fixed number of randomly selected combinations. Its computational cost is therefore controlled by the selected number of candidates rather than the total size of the search space.
- For a large search space, Randomized Search is generally more practical because it can explore a broad range of configurations with a fixed computational budget.

6. Recommend the best MLP configuration.

Based on the reported results, the baseline model is preferable because it achieved the higher testing accuracy of 0.8794 compared with 0.8747 for the optimized configuration. The baseline uses two hidden layers with

128 and 64 neurons, ReLU activation, and the Adam optimizer. Further optimization could be performed with a larger search budget and a more representative validation procedure.

12. Conclusion

In this experiment, a TensorFlow/Keras MLP was trained on the Fashion-MNIST dataset and hyperparameters were tuned using Randomized Search with 5-fold cross-validation. The baseline model achieved a testing accuracy of 0.8794, while the selected optimized configuration achieved 0.8747. The results demonstrate the importance of preprocessing, model architecture, and hyperparameter selection, while also showing that optimization does not necessarily improve performance when the search budget and sampled configurations are limited.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.